

SmartHarvest - EO-driven Vineyard Monitoring

Project Proposal Document

1. Project Summary

Project title: SmartHarvest: data-driven precision agriculture

Team: Lorenzo Di Bernardo, Giovanni Mason, Andrea Leonesio, Andrea Puccio, Bruno Trevisan

Project Duration: December 2025 - January 2026

Course: Data-driven Systems Engineering (MLOps)

High-level definition

SmartHarvest is a low-opex, scalable MLOps platform for vineyard monitoring that processes open Earth Observation satellite time series and static geospatial layers to detect and visualize early anomaly signals and within-parcel maturity non-uniformity, without requiring laboratory analysis or mandatory in-field sensors.

Objective (SMART)

Within the 1^o month of pilot usage, deliver an on-demand analysis workflow that produces conservative anomaly detection outputs and maturity uniformity monitoring, enabling agronomists and vineyard managers to prioritize scouting on grapevines at risk using satellite-derived signals, with explicit observation reliability reporting.

MVP Scope

Use case: detect intra-parcel anomalies and spatial non-uniformity that may indicate uneven ripening or stress patterns, and present results in a map-first interface anchored to the latest available EO acquisition, with the ability to review historical runs over time.

Data sources: open EO satellite observations and static topography layers; optional data (meteorological data/IoT/ground-truth) supported only when provided (out of core MVP).

Core requirements: single-user workspace with parcel management (save and analyze multiple parcels per user); on-demand run execution anchored to a target acquisition (default: latest available); output generation (anomaly heatmap, macro report, reliability factor/reliability report); export of results as PDF/CSV/GeoJSON; storage of run metadata for reproducibility and auditability; UI support for listing and opening past runs for the same parcel (timeline/history navigation).

Constraints: unsupervised-first due to scarce labels; cloud coverage and acquisition quality limitations; scalable architecture to support multiple users and parcels while keeping operational costs low.

Key stakeholders

- Primary: agronomists, vineyard managers, viticulturists.
- Future extensions: insurers/banks/PA/consortia.

Budget and resources

Reliance on open data sources; low operational cost target.

Expected outcomes

A reproducible end-to-end monitoring workflow that improves current vineyard monitoring practices by providing:

- (i) heat maps for spatial anomaly visualization (map-first, updated at each available overpass),
 - (ii) maturation uniformity monitoring reports via anomaly prevalence and spatial distribution,
 - (iii) explicit observation reliability indicators that prevent over-trusting low-quality satellite observations, while enabling historical review of field evolution across successive acquisitions.
-

Success criteria

SC1 - End-to-end run reliability

On a predefined test set of 20 run requests, at least 95% of valid run requests reach a terminal state without crashing and produce a complete run bundle of outputs.

A run request is valid if AOI geometry passes schema validation and the run request is syntactically correct. Terminal state includes SUCCESS, FAILED, or LOW_CONFIDENCE_DONE.

SC2 - Output and export completeness

Every run that reaches a terminal state must produce a run bundle containing:

1. Anomaly heatmap (acquisition-anchored)
2. Macro anomaly report
3. Coverage-reliability factor

plus exports in PDF, CSV, GeoJSON.

SC3 - Conservative confirmation policy

For every run, 100% of anomalies labeled as confirmed must satisfy the ensemble confirmation rules and must not be confirmed under LOW_CONFIDENCE_DONE.

For acquisition-anchored runs, confirmation rules are evaluated using configured consistency checks (e.g. spatial coherence and cross-feature coherence) and, where applicable, persistence across successive acquisitions.

SC4 - Missing data robustness

If observation adequacy is below a defined threshold for the target acquisition, the system must:

- set terminal state to a low confidence label,
- produce the 3 core outputs with exports,
- not label any anomaly as confirmed, and expose a low-confidence trigger in report and UI.

SC5 - Usability baseline

In a guided test session using the provided “demo command”, a new user, within ≤ 30 minutes and

without developer intervention, can: create account, save/import a parcel, start a run. view results and download exports.

2. Requirements

User Requirements (UR)

- **UR-01:** The user shall create/select a parcel, submit an on-demand run for the latest available EO acquisition (default behavior), monitor status, view results, and download exports.
- **UR-02:** The user shall view run outputs in a map-first dashboard (anomaly heatmap, macro report, reliability indicator), and switch between available map layers for the selected run.
- **UR-03:** The user shall download exports in PDF, CSV, and GeoJSON formats.
- **UR-04:** The user shall be informed when results are low-confidence due to insufficient EO observation adequacy or poor EO quality for the target acquisition, and the UI shall avoid presenting anomalies as confirmed under low-confidence conditions.
- **UR-05:** The user shall browse historical runs for a parcel via a date/timeline selector and open past results to review field evolution over time.

External Interface Requirements

User Interface (UI)

- **UI-01:** The UI shall allow parcel creation/import (GeoJSON/Shapefile) and provide validation feedback against the AOI contract.
- **UI-02:** The UI shall allow run submission for the latest available acquisition (and, if enabled, a selected past acquisition) and status tracking until it reaches a terminal state.
- **UI-03:** The UI shall present outputs in a map-first layout and enable download of PDF/CSV/GeoJSON exports for the currently selected run.
- **UI-04:** The UI shall provide a run history view (list and/or timeline) that allows selecting a run by acquisition date and switching the displayed outputs accordingly.
- **UI-05:** The UI shall surface observation reliability consistently (indicator and/or reliability map) and clearly distinguish LOW_CONFIDENCE_DONE runs from SUCCESS runs.

API Interfaces (API)

- **API-01:** The API shall accept a run request for a parcel and a target acquisition selector (default: latest available) and return a unique run_id.
- **API-02:** The API shall return run status and a structured failure reason if FAILED, and a structured low-confidence reason when status is LOW_CONFIDENCE_DONE.
- **API-03:** The API shall return run outputs (including artifact references for heatmap, macro report, reliability report, and exports), reliability indicator, and core metadata for a given run_id.
- **API-04:** The API shall list available runs for a parcel, including acquisition date/scene identifier, terminal status, reliability summary, and references required for UI history/timeline navigation.

Data Requirements (DATA)

- **DATA-01:** AOI inputs shall conform to a defined GeoJSON schema and validation rules.
- **DATA-02:** Each run shall produce a run bundle with mandatory outputs, exports, stable naming, and acquisition anchoring (acquisition date and/or scene identifier) sufficient for traceability and reproducibility.
- **DATA-03:** Exported GeoJSON shall be syntactically valid and include required attributes, including run_id, aoι_id, acquisition date/scene identifier, status, reliability_factor, confidence_label, and macro summary fields.

System Functional Requirements (FR)

- **FR-01:** The system shall authenticate users for UI/API access.
- **FR-02:** The system shall support parcel creation/import with validation.
- **FR-03:** The system shall allow saving and retrieving parcels per user across sessions.
- **FR-04:** The system shall create a run record and unique run_id on user submission.
- **FR-05:** The system shall manage run statuses: QUEUED, RUNNING, SUCCESS, FAILED, LOW_CONFIDENCE_DONE.
- **FR-06:** The system shall ingest required EO observations and static layers for the AOI and the target acquisition (default: latest available), identified by acquisition date and/or scene identifier; optional minimal contextual acquisitions may be retrieved when required for stabilization/tracking, without being a user-defined time window.
- **FR-07:** The system shall apply preprocessing and QA prior to analysis.
- **FR-08:** The system shall compute core indices and features required by the kernel.
- **FR-09:** The system shall compute a pixel-level anomaly score without ground-truth labels.
- **FR-10:** The system shall compute a coverage-reliability factor for every run.
- **FR-11:** The system shall confirm anomalies only when configured consistency/ensemble rules are satisfied and observation reliability is adequate, LOW_CONFIDENCE_DONE runs shall not present anomalies as confirmed.
- **FR-12:** For each terminal run, the system shall generate the following mandatory outputs: anomaly heatmap, macro report, reliability factor/reliability report.
- **FR-13:** The system shall generate PDF/CSV/GeoJSON exports for each terminal run.
- **FR-14:** The system shall store run metadata sufficient for reproducibility and auditability, including acquisition anchoring fields, quality/reliability summaries, versions, parameters, and artifact references inside the run bundle.
- **FR-15:** FAILED runs shall include structured error information (stage, reason).
- **FR-16:** The system shall list runs for a parcel (ordered by acquisition date), enabling UI history/timeline navigation and retrieval of outputs/exports for past runs.
- **FR-17:** The system shall persist a tracking representation across successive runs that supports zone/cluster evolution summaries where applicable.

Non Functional Requirements (NFR)

- **NFR-01:** On 20 valid runs, ≥95% reach a terminal state and produce a complete run bundle.
- **NFR-02:** Every terminal run produces the run bundle with 3 mandatory outputs and exports.
- **NFR-03:** No confirmed anomaly violates the ensemble/consistency rules.
- **NFR-04:** Low reliability or insufficient observation adequacy triggers LOW_CONFIDENCE_DONE and prevents confirmed anomalies.

- **NFR-05:** A guided user completes the workflow ≤ 30 minutes.
- **NFR-06:** Runs emit logs/metrics sufficient to compute success rate, failure-by-stage counts, and low-confidence incidence.

The table below provides traceability from each Success Criterion to the key requirements, along with the corresponding deliverables and milestone gates used for verification, which are detailedly described below.

Success Criteria	Requirements Summary	Deliverable	Milestone
SC1 - Run reliability ($\geq 95\%$ terminal states)	FR-04/05/15, API-01/02, NFR-01/06, FR-14	D1 (Data Backbone), D3 (Serving Layer API), D5 (CI/CD)	M3 (Run lifecycle), M5 (Monitoring)
SC2 - Output and export completeness (per terminal run)	FR-12/13/14, UI-03, API-03, DATA-02/03, NFR-02	D2 (ML Kernel & outputs), D5 (CI/CD)	M2 (Outputs & exports), M4 (CI gates)
SC3 - Conservative confirmation policy (no violations)	FR-11, NFR-03, NFR-04 / UR-04	D2 (ML Kernel & outputs), D5 (CI/CD)	M4 (Structural gate), M5 (Policy completed)
SC4 - Missing data robustness (LOW_CONFIDENCE_DONE and no confirmations)	FR-05/10/11/12/13, UR-04, API-02, NFR-04	D1 (Data Backbone), D2 (ML Kernel & outputs), D4 (Web App)	M2 (Reliability factor exists), M3 (Status in UI), M5 (Conservative policy)
SC5 - Usability (≤ 30 min guided session)	UR-01/02/03/04, UI-01/02/03, API-01/02/03, NFR-05, FR-01	D4 (Web App), D3 (Serving Layer API)	M3 (End-to-end workflow)

All structured requirements are specified in the SSD maintaining full traceability.

3. Data Sources and Data Requirements

3.1 Earth Observation

For the MVP, SmartHarvest relies primarily on open Earth Observation (EO) data to support the unsupervised monitoring workflow. The baseline EO source is **Sentinel-2 L2A**, used to derive vegetation and water-related indices (e.g. NDVI/NDRE/NDMI) and their temporal dynamics.

EO data are complemented by static geospatial layers describing field morphology (**SRTM DEM**) from which slope, aspect, and simple proxies of insolation can be derived.

In addition, the system may integrate complementary sources that strengthen robustness under different environmental conditions: Sentinel-1 SAR backscatter, useful when optical data are degraded by clouds; thermal observations from Landsat (8/9) TIR/LST and ECOSTRESS LST to capture

temperature-driven stress signals, and ERA5-Land variables (soil moisture and temperature and climatic context).

Module	Sentinel-2 L2A	SRTM DEM	Sentinel-1
Provider	ESA Copernicus (Sentinel-2A/2B)	NASA, USGS, SRTM mission	ESA Copernicus (Sentinel-1A/1B)
Data type	Optical multispectral (surface reflectance)	Static topography (DEM)	Radar (C-band SAR)
Variables used/features	Band reflectances (NDVI, NDRE, NDMI, time-series stats)	Elevation (slope, aspect, insolation proxy)	Backscatter (VV/VH) time series, texture proxies
Typical spatial resolution	10 m (some bands 20-60 m)	~30 m (1 arc-second)	~10 m
Typical temporal resolution	~5 days (sentinel 2 A+B combined), cloud-limited	Static	~6-12 days (orbit/geometry dependent), cloud-independent
Coverage	Global	Near-global (no poles)	Global
Utility	Core parcel-scale canopy vigor, chlorophyll, moisture proxies	Explains terrain-driven variability; stabilizes interpretation	Robust signal under clouds; complementary canopy structure-moisture info
Main limitations	Cloud gaps; mixed pixels at boundaries	Coarser than S2; static	Speckle; geometry effects; less intuitive interpretation

Module	Landsat Thermal / LST	ECOSTRESS	ERA5-Land
Provider	NASA, USGS (Landsat 8/9)	NASA / ISS	ECMWF Copernicus
Data type	Thermal infrared (surface temperature proxy)	Thermal infrared (LST/potential ET proxies)	Reanalysis (model-obs assimilation)
Variables used	LST or thermal bands (thermal stress proxies)	LST	Soil moisture/temperature, precipitation, radiation (context)
Typical spatial resolution	Thermal native coarser (often 100 m+), commonly distributed on 30 m grids	~70 m (order of magnitude; product-dependent)	~0.1° (~9 km)
Typical temporal resolution	16 days per satellite (8/9 combined ~8)	Variable sampling (ISS orbit), different times	Hourly (aggregatable weekly)

	days), cloud-limited	of day	
Coverage	Global	Large portions of globe (non-uniform)	Global
Utility	Thermal stress signal complementary to optical	High-value thermal snapshots, often different time-of-day	Climatic/soil context; supports interpretation and robustness
Main limitations	Coarse; infrequent; cloud sensitivity	Irregular revisit; availability not guaranteed; cloud sensitivity, SF only	Very coarse; aggregation needed

3.2 Data frequency and analysis windows

The platform is designed to accommodate refreshing whenever the user explicitly requests a run, in line with his own monitoring needs. Importantly, the MVP does not enforce an automatic run schedule: analyses are executed on-demand.

Users select the parcels of interest and request an analysis anchored to a target EO acquisition, that is by default the latest available acquisitions for the parcel.

The MVP does not require the user to specify temporal ranges or phenological windows to produce outputs. When historical review is needed, users navigate previously executed runs through a date/timeline selector.

3.3 Data quality constraints and reliability reporting

Because optical EO observations are frequently impacted by cloud coverage and missing data, SmartHarvest explicitly models observation reliability at the acquisition level.

Each run must compute a coverage-reliability factor summarizing the adequacy of the available EO observations for the target acquisition and parcel.

When observations are insufficient, the system must avoid overconfident conclusions by triggering LOW_CONFIDENCE_DONE and adopting a conservative no-decision behavior, such as producing the mandatory outputs and exports while preventing anomalies from being presented as confirmed, rather than forcing interpolated or unjustified results.

Geospatial harmonization is treated as a first-class requirement: the system must apply consistent rules for CRS alignment and resampling across layers, distinguishing between continuous layers and categorical layers.

Finally, each run must be fully traceable: the platform records the data sources used, acquisition date and/or scene identifier, any effective acquisition range used internally, key parameters, and the versions of datasets and artifacts that generated the result, supporting reproducibility and auditability.

4. User workflow

The MVP workflow is kept intentionally essential: the user defines or selects parcels, runs analyses on demand, and retrieves results as maps and downloadable artifacts.

The workflow is acquisition-anchored: each run corresponds to a specific EO acquisition (default: the latest available Sentinel-2 overpass for the selected parcel), producing a deterministic snapshot of field status that can be reviewed over time through run history navigation.

1. Area of Interest Definition

The user either imports a parcel boundary (GeoJSON/Shapefile) or draws it directly on the map interface, and then saves it for future runs. The system validates the geometry and stores it as canonical GeoJSON to ensure stable clipping, reproducible processing, and consistent parcel retrieval across sessions.

2. Run Execution

The user initiates an on-demand run for the selected parcel, anchored to the latest available acquisition. The platform performs EO ingestion and preprocessing for the target acquisition, computes features and anomaly outputs, and exposes run status until completion.

3. Results Visualization

The user accesses the run outputs in a map-first dashboard and can browse historical runs through a date/timeline selector to review field evolution across successive acquisitions. Each run returns three mandatory products:

- 1) an anomaly heatmap where pixels are colored according to the detected deviance score for the target acquisition,
- 2) a macro report summarizing the percentage of anomalous pixels over the parcel,
- 3) a coverage-reliability factor that contextualizes how much the result can be trusted given observation quality.

The platform does not provide “possible cause” narratives (e.g., water stress vs disease) in the MVP, because such causal interpretations would be unreliable without consistent ground truth and expert involvement.

4. Export and Reporting

Results are made available for download as PDF, CSV, and GeoJSON for each terminal run.

As now, the system does not rely on a mandatory human validation loop to define success. If user feedback or ground truth is available, it may be stored as a future-enabling capability, but it is not required to operate the system and MVP-core.

6. Deliverables

D1 - Reproducible Data Backbone

A data pipeline that retrieves and preprocesses EO data for a selected parcel (AOI) and time window, producing harmonized, quality-controlled time series and structured feature outputs.

Includes:

- Sentinel-2 L2A ingestion (AOI clipping, time-window selection).
- Other satellites EO ingestion.
- Preprocessing and QA: cloud masking constraints, valid-pixel statistics, CRS alignment/resampling rules, mixed-pixel awareness and boundary handling.
- Structured feature store outputs and a dataset population procedure for time-series readiness.

D2 - ML Kernel Baseline and Outputs

A working unsupervised, acquisition-anchored detection kernel that computes per-pixel deviance scores and generates the three mandatory run outputs in a stable, testable format.

Includes:

- Kernel input specification (with expected features, grid alignment, shapes, and normalization assumptions).
- Spatial anomaly scoring for a target acquisition (baseline deviance score; extensible to multi-sensor).
- Optional zone/cluster representation for denoised visualization and tracking summaries across successive acquisitions (UI-enabling).
- Coverage-reliability factor computation and thresholding per run, with conservative LOW_CONFIDENCE_DONE / no-decision behavior when observation adequacy is insufficient (NFR-02, NFR-04).
- Generation of the three mandatory outputs (SC2 / FR-12)

D3 - Minimal Serving Layer (API)

A minimal backend interface enabling the platform to be used as an on-demand system.

Includes:

- API endpoints supporting run lifecycle (FR-04): create run, query run status, retrieve results, retrieve artifacts/exports.
- Run status model aligned with the system lifecycle (queued/running/success/failed/low-confidence).
- Result retrieval contract compatible with UI integration.

D4 - Web App

A minimal backend interface enabling the platform to be used as an on-demand, acquisition-anchored system.

Includes:

- API endpoints supporting the run lifecycle (FR-04): create run, query run status, list runs for history navigation, retrieve results, retrieve artifacts/exports.
- Run status model aligned with the system lifecycle (QUEUED/RUNNING/SUCCESS/FAILED/LOW_CONFIDENCE_DONE), including structured failure reasons for FAILED and structured low-confidence rationale for LOW_CONFIDENCE_DONE (FR-05, FR-15).
- Result retrieval contract compatible with UI integration: returns acquisition anchoring (acquisition_date and/or scene_id), reliability indicators, and stable artifact references/keys for mandatory outputs and exports, enabling map-first visualization and timeline-based browsing.

D5 - CI/CD and Packaging

A reproducible build and run packaging strategy and CI pipeline that provides regression safety for an on-demand, acquisition-anchored MVP.

Includes:

- Container build (Docker) and a single demo command that, on a clean machine, runs an end-to-end execution and produces a complete run bundle (artifacts + exports + metadata).
- CI pipeline (GitHub Actions) running:
 - lint/style checks,
 - smoke tests (pipeline execution on a test AOI),
 - SC2 regression checks (mandatory artifacts and exports exist; stable logical keys; acquisition anchoring fields present),
 - SC3 structural gate (policy compliance invariant),
 - GeoJSON validity checks for exports.

D6 - Monitoring Baseline and Run Auditability (health, quality, completeness)

A meaningful monitoring layer focused on operational trustworthiness for an unsupervised, acquisition-anchored system.

Includes:

- Run health metrics: success rate, terminal-state distribution, failure-by-stage, latency/runtimes (eg. p95).
- Data quality metrics: observation adequacy signals for the target acquisition (valid-pixel ratio, cloud-masked ratio where applicable, missingness/coverage summaries), coverage-reliability distributions, and LOW_CONFIDENCE_DONE rate trends over time (SC1, SC4).

- Artifact completeness and export success signals: presence of mandatory artifacts/exports per terminal run, export validation outcomes (CSV/GeoJSON), and download/serve errors where applicable (SC2).
- Run auditability: stage-level structured errors for FAILED runs, and run metadata fields (acquisition anchoring, data provenance, parameters, versions, artifact references) enabling debugging and post-mortem analysis.

D7 - Final Documentation

A complete documentation package that makes the system understandable, runnable, and verifiable by an external evaluator.

Includes:

- Final project proposal and SSD
 - Architecture description (components, interfaces, data flow).
 - User guide (SC5).
-

7. Milestones

M0 - Scope baseline, requirements freeze, and project hygiene

Milestone 0 sets the project: scope and success criteria, the repository is runnable, and the team sketches MVP boundaries and the execution plan.

Gate to pass: frozen proposal sections (MVP scope, Success Criteria SC1-SC5), initial repo structure, environment setup instructions.

Sprint: Sprint 1

M1 - Data backbone reproducible (acquisition-anchored)

Milestone 1 is passed when EO access, preprocessing, and QA are reproducible and produce stable run artifacts for at least one test parcel, anchored to a target acquisition (latest available). This milestone establishes ingestion modularity, deterministic preprocessing, and run-oriented artifact writing so metadata, CI, and monitoring can attach smoothly.

Gate to pass: one reproducible run on a test parcel that retrieves EO for a target acquisition, applies key preprocessing rules (masking, alignment to the 10 m reference grid, valid-pixel accounting), computes acquisition-level adequacy statistics.

Sprint: Sprint 2 (pipeline sketch and ingestion), Sprint 3 (QA mechanics and alignment checks), Sprint 4 (provenance and metadata storage).

M2 - ML Kernel baseline

Milestone 2 does not claim the full anomaly detection logic is complete, but this gate is passed when a first working unsupervised kernel baseline is available and each terminal run produces the three mandatory outputs in a consistent and testable shape:

- anomaly heatmap (baseline deviance score),
- macro report (% anomalous pixels + required summary fields, acquisition-anchored),
- coverage-reliability factor (with LOW_CONFIDENCE_DONE behavior available when adequacy is insufficient).

Gate to pass: given a parcel and a target acquisition selector, a run completes and generates the three mandatory outputs plus exports (PDF/CSV/GeoJSON) in the run bundle, with acquisition anchoring fields present in artifacts and metadata.

Sprint: Sprint 5, 6

M3 - Serving layer and run lifecycle

Milestone 3 is passed when a user can execute the on-demand workflow end-to-end, manage multiple parcels, and observe a real run lifecycle (QUEUED/RUNNING/SUCCESS/FAILED/LOW_CONFIDENCE_DONE). The API serves results through stable contracts, and the UI supports map-first visualization and historical navigation (list/timeline of runs).

Gate to pass: user can select/save a parcel, trigger a run for the latest acquisition, observe status transitions, view results in the map-first dashboard, browse available past runs via run listing/timeline, and download artifacts/exports, consistently with SC5 and the API/UI contracts.

Sprint: Sprint 7, 8.

M4 - Deployment repeatability and CI (regression gates)

Milestone 4 is passed when the project is containerized, execution is reproducible on a clean machine, and CI enforces regression checks aligned with the contracts and success criteria.

Gate to pass: Docker build and reproducible demo command; GitHub Actions pipeline that runs lint/tests and includes regression checks for SC2 output completeness, SC3 policy invariants and GeoJSON validity/schema-required fields.

Sprint: Sprint 7

M5 - Monitoring baseline and conservative policy completion

Milestone 5 is passed when the system exposes monitoring signals for operational trustworthiness (run health, latency, failure-by-stage, LOW_CONFIDENCE_DONE rate trends, artifact completeness/export validity), end-to-end tests are executed, and the conservative confirmation/no-decision policy is completed and aligned with SC3-SC4.

Gate to pass: monitoring summaries derived from run events/metadata, end-to-end test evidence over a test campaign, final MVP script/demo command, and a completed conservative policy layer whose invariants match SC3 and SC4 (including explicit low-confidence rationale in reports/UI/exports).

Sprint: Sprint 8

8. Work Breakdown Structure (WBS)

1. Project Management and Controls

- 1.1 Scope baseline**
- 1.2 Risk Analysis and Mitigation**
- 1.3 Milestone gate reviews**
- 1.4 Team coordination**

2. Data Layer

2.1 AOI data contract (GeoJSON schema, required fields, validation rules)

2.2 EO ingestion connectors

- 2.2.1 Sentinel-2 L2A retrieval (time window and AOI clipping)
- 2.2.2 Sentinel-1 SAR ingestion (if enabled)
- 2.2.3 Landsat thermal / LST ingestion (if enabled)
- 2.2.4 ECOSTRESS LST ingestion (if enabled)
- 2.2.5 ERA5-Land context retrieval and aggregation (if enabled)
- 2.2.6 SRTM DEM retrieval (AOI clipping; static caching strategy)

2.3 Preprocessing and QA

- 2.3.1 Cloud masking and valid pixel statistics (per acquisition and aggregated)
- 2.3.2 Geospatial harmonization (CRS alignment and resampling policy)
- 2.3.3 Missingness rules (minimal acquisitions; no forced interpolation beyond policy)

2.4 Feature calculation

- 2.4.1 Vegetation/water indices (NDVI/NDRE/NDMI)
- 2.4.2 Terrain derivatives (slope, aspect, insolation proxy)

2.5 Dataset and artifact versioning

- 2.5.1 Run data manifest (sources, dates, scene IDs, coverage stats)
- 2.5.2 Feature store outputs (structured outputs saved per run)

3. ML Kernel Layer

3.1 Kernel input specification (features required, normalization rules)

3.2 Spatial anomaly evidence

3.3 Temporal anomaly evidence

3.4 Ensemble confirmation policy

3.4.1 Multi-index coherence rule implementation

3.4.2 Cross-acquisition consistency/persistence rule implementation

3.4.3 Policy outputs: candidate vs confirmed vs informational/no-decision

3.5 Coverage-reliability factor

3.5.1 Reliability definition and components (coverage ratio, valid pixels, acquisition count)

3.5.2 Thresholding logic (low-confidence / no-decision behavior)

3.6 Outputs generation

3.6.1 Pixel-level anomaly heatmap generation (deviance score)

3.6.2 Macro report computation (% anomalous pixels; summary fields)

3.6.3 Output packaging for UI and exports

3.7 Traceability and metadata storage

3.7.1 Store kernel version and policy version per run

3.7.2 Store run parameters (windows, thresholds, feature set)

4. Backend and API Layer

4.1 Run orchestration service

4.1.1 Create run (submit parcel(s), time window, config)

4.1.2 Run status lifecycle (queued/running/success/failed/low-confidence)

4.1.3 Result retrieval endpoints (heatmap and reports and metadata)

4.1.4 Run listing endpoint for parcel history (dates/status/reliability)

4.2 Artifact persistence and registry (store paths, checksums, availability)

4.3 Export services

4.3.1 PDF report export

4.3.2 CSV export

4.3.3 GeoJSON export and basic validity checks

4.4 Error handling and observability hooks (structured errors by stage for monitoring)

5. Frontend (Web App / Dashboard)

5.1 Parcel definition UI

5.1.1 Draw parcel on map

5.1.2 Import GeoJSON/Shapefile and validation feedback

5.2 Run configuration UI (time range and window selection; run submit)

5.3 Run status UI (progress indicator; error states; retry where allowed)

5.4 Map-first results UI

- 5.4.1 Heatmap visualization (deviance score rendering)
- 5.4.2 Macro report view (% anomalous pixels)
- 5.4.3 Coverage-reliability display (explicit low-confidence/no-decision states)
- 5.4.4 History/timeline UI (select run by acquisition date; browse evolution)

5.5 Downloads UI (PDF/CSV/GeoJSON access; artifact availability)

6. CI/CD, Packaging, Monitoring

6.1 Repository structure and code standards (linting, folder layout, dependency)

6.2 Containerization and environment reproducibility

- 6.2.1 Build container image(s)
- 6.2.2 Runtime configuration management (env vars, secrets placeholders)

6.3 Continuous Integration (baseline checks)

- 6.3.1 Smoke tests: pipeline runs on a test parcel
- 6.3.2 Regression tests: SC2 output completeness
- 6.3.3 Policy tests: SC3 (no confirmed anomaly violates ensemble rules)
- 6.3.4 Export schema tests: GeoJSON validity and required fields

6.4 Continuous Delivery

- 6.4.1 Deploy script / docker-compose template
- 6.4.2 Repeatable deployment instructions

6.5 Monitoring baseline

- 6.5.1 Run health metrics (success rate, failure by stage, latency)
- 6.5.2 Data quality metrics (coverage, valid pixels, cloud-masked ratio, reliability trend)
- 6.5.3 Artifact completeness and export success metrics
- 6.5.4 Minimal dashboard or logs summary for demo evidence

7. Documentation and Final Submission

7.1 Project Proposal document finalization

7.2 System Specification document finalization

7.3 Operational Governance and versioning document finalization

WBS graphical notation:

<https://drive.google.com/file/d/1Dh8Ad42lYsDIQwwQjllKsMdax8emxKB4/view?usp=sharing>

9. Sprint Plan

Development is executed using an Agile approach tracked on the Trello board (Sprint 1-Sprint 7). The sprint workload is intentionally asymmetric: early sprints are dominated by setup, alignment, and data-backbone hardening in order to unblock later work, while the final sprints concentrate integration, reproducibility and traceability, CI gates, monitoring, and demo readiness.

In addition, the plan explicitly accounts for a mid-project pivot: the original time-series anomaly detection direction proved impractical under the available constraints (in particular, difficulties in operationally populating and maintaining the expected time-series database substrate), therefore the ML kernel was re-scoped toward an unsupervised clustering-based approach. As a consequence, Sprint 6 is dedicated to repopulating the database coherently with the clustering kernel needs, and Sprint 7 consolidates multiple parallel integration and hardening tasks required for a complete MVP.

Sprint 1

Sprint 1 establishes the foundation: domain knowledge consolidation, Agile planning artifacts and sprint outline, local environment setup, and the start of documentation and versioning control practices that continue across all sprints. The focus here is to freeze scope assumptions early and align the team on success criteria, contracts, and expected evidence.

Sprint 2

Sprint 2 builds the first technical backbone and shared contracts: a pipeline sketch that defines the run flow end-to-end at a high level and the initial feature store structure. A UI baseline skeleton is initiated to anticipate the on-demand workflow, while the analytics direction converges on an unsupervised approach consistent with label scarcity and EO uncertainty constraints.

Sprint 3

Sprint 3 expands ingestion architecture toward multi-source readiness and introduces explicit data-quality mechanics. Work focuses on a modular ingestion library structure, explicit thresholds and PASS/WARN/FAIL outcomes for data adequacy, and key EO aspects such as mixed pixels and cloud masking constraints. In parallel, scripts for core agronomic indicators over the feature store are drafted to prepare the feature layer for the kernel.

Sprint 4

Sprint 4 concentrates on dataset maturity and on preparing the analytical substrate required by the ML kernel. The sprint covers interpolation analysis preparation, the comparison between static and dynamic database strategies, and procedures to populate the time-series dataset in a reproducible way, together with the first formal definitions of metadata fields needed for traceability and monitoring. CI/CD planning is consolidated at this stage to ensure that subsequent components can be integrated under automated verification.

Sprint 5

Sprint 5 shifts from data collection only toward MVP trustworthiness and service readiness. It includes initial implementation of the coverage-reliability factor, agronomic metrics scripting, and first versions of interpolation and time-series analysis. In parallel, the sprint initiates the serving layer through API endpoints for run creation/status/results/artifacts, advances the UI baseline and interaction flow, starts UI-API integration, introduces a first CI/CD implementation draft, and brings metadata storage to an initial operational version to support reproducibility and traceability.

Sprint 6

Sprint 6 is explicitly planned as a pivot and consolidation sprint driven by feasibility and robustness

considerations. Given the difficulties encountered with the original time-series anomaly detection direction, the ML kernel is re-scoped toward an unsupervised clustering-based approach. Accordingly, Sprint 6 is dedicated to repopulating the database coherently with the new kernel requirements, restructuring the dataset population procedure to produce clustering-ready inputs, and updating the metadata layer so that the new analytical pathway remains auditable and reproducible.

Sprint 7 and subsequent

Sprint 7 is the final integration and delivery packaging sprint. It is planned to concentrate multiple concurrent workstreams: completion of the clustering implementation in the ML kernel, completion of post-processing through the ensemble/conservative policy layer, integration of an asynchronous run orchestrator with explicit stage statuses, implementation of export services (PDF/CSV/GeoJSON) together with an artifact registry, and continued UI advancement to ensure that the single-user workflow is smooth and consistent with the acquisition-anchored run model. In addition, Sprint 7 plans container build and a reproducible “demo command”, implementation of monitoring signals aligned with the success criteria, and end-to-end software tests that validate the run bundle completeness and policy invariants. Documentation review and polishing are treated as integral to this sprint in order to deliver a coherent, inspectable submission package.

A dashboard and a Gantt-style overview of tasks and dependencies are maintained in the Trello team board to support coordination and to provide governance evidence for evaluation.

Trello Dashboard and Gantt-Chart link:

<https://trello.com/invite/b/695024066922412744bc6b63/ATTI53c98646724c9e11f952a40a3d709550F37A28C5/smart-harvest>

10. Definition of Ready (DoR)

A user story/task is Ready when:

1. **Traceability is explicit:** references at least one Deliverable, WBS node, and one SC/FR/NFR it contributes to.
2. **Acceptance criteria are measurable:** written as pass/fail checks.
3. **Inputs/outputs are defined:** expected inputs, produced artifacts/API responses, naming/schema expectations.
4. **Dependencies are resolved:** required modules and resources exist (endpoints, storage paths, sample parcel, test window) or the block is explicitly pointed out.
5. **Test plan is stated:** minimum tests (unit/integration/smoke) and how to run them.

11. Definition of Done (DoD)

A user story/task is Done when:

1. **Implemented and integrated:** code is merged (or ready to be merged), contracts/schemas not broken.
2. **Reproducible:** one single command to run/verify and environment assumptions documented (deps/config/env vars).
3. **Tests pass:** required tests exist and pass (unit/integration/smoke). If tied to SC2/SC3, include explicit regression checks; exports validated (GeoJSON schema/required fields).
4. **Traceability evidence exists:** run metadata includes data sources, time window, AOI hash, parameters (thresholds/policy/feature set), code version, and container tag if used.
5. **Observability baseline:** structured errors (stage and reason), minimal logs for run status and failures-by-stage.
6. **Review:** at least one teammate review.